

トークンの系列の扱い

自然言語処理＞モデル＞系列性



どうやってトークンの並びの系列をモデルで扱うか？

- テキストデータを処理する上での考慮事項は主に2つある
 - 離散的：数値ではなく「文字」という離散的なものを入出力できる必要がある
 - ▶ そのままでは，モデルの学習はおろか，四則演算などの計算すらできない
 - **可変長**：入力される文字数が予め決まっていない
 - ▶ そのまま順伝播型ニューラルネットなどで扱うことは出来なさそう
- ここでは，トークンの**系列**の扱い方の話を解説
 - トークンの並びには意味がある． また，系列長さも不定な入出力になる．

自然言語処理＞モデル＞単語の意味の文脈依存性

単語の意味は，文脈によって大きく変わる



鍵 を かける



電話 を かける

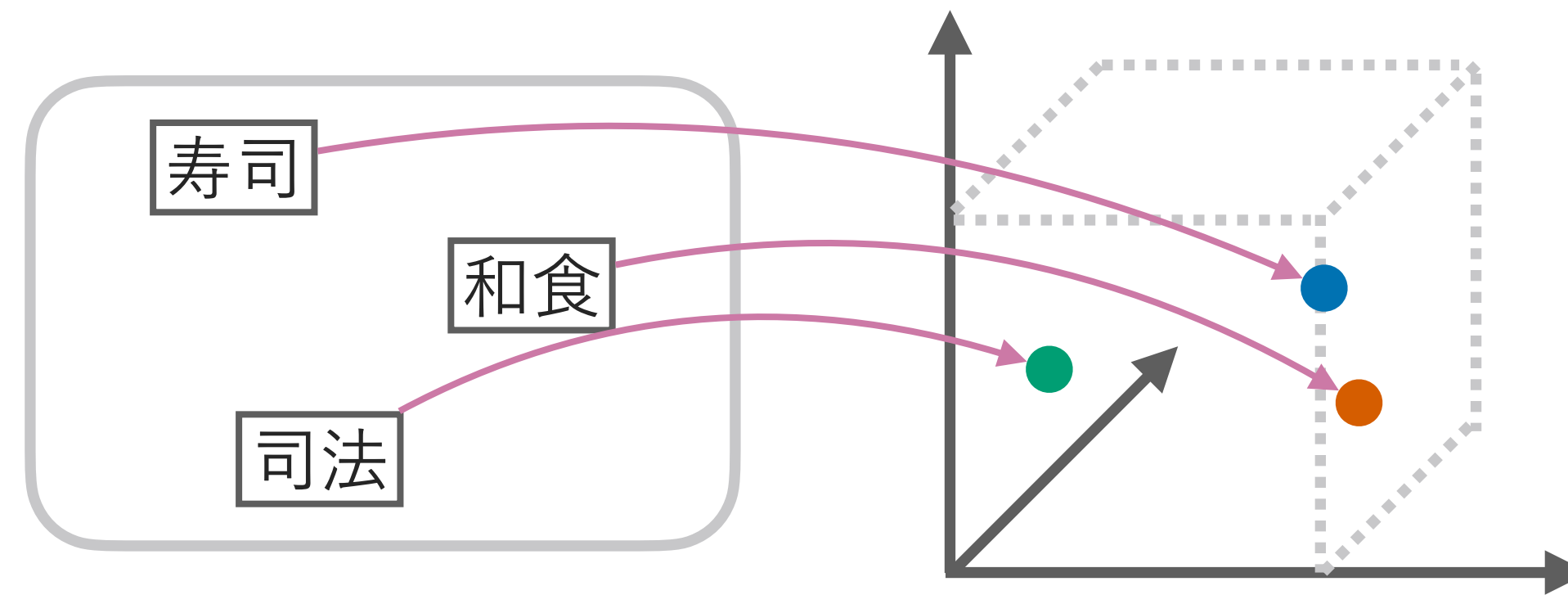


etc.

自然言語処理＞モデル＞文脈化単語埋め込み

系列を扱う中では，単語埋め込みも文全体を見て決められるとなお良い

- ここまでの単語埋め込みは，1単語に1ベクトルと，辞書的に使われるものだった

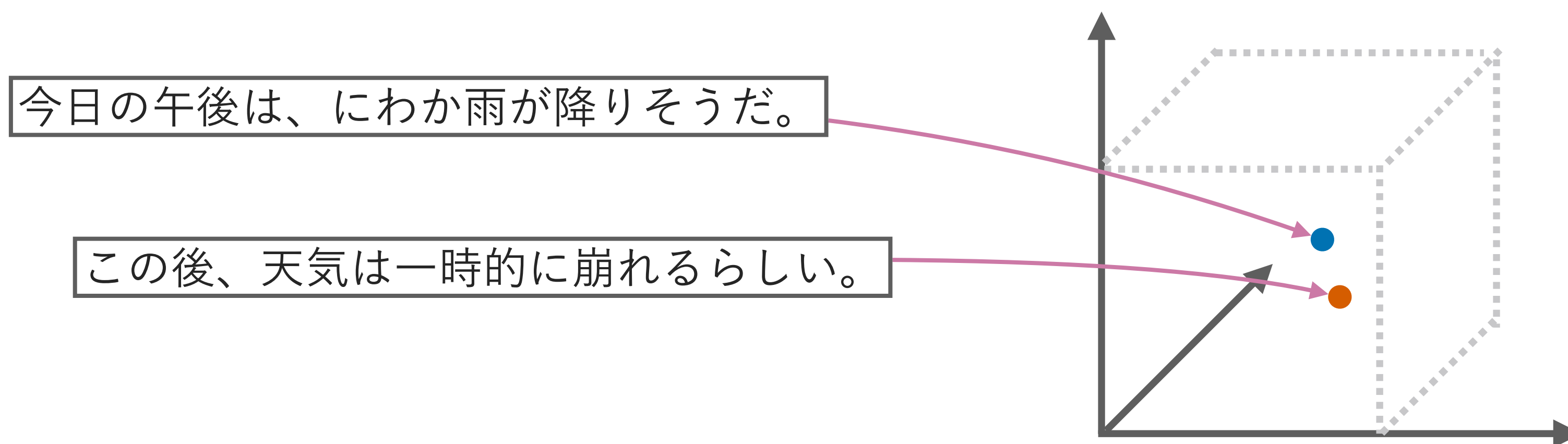


- 実際には，多くの単語は，文脈によって意味が変わるが，通常の単語埋め込みでは，どちらの「今日」も同じベクトルにしてしまう
 - 今日は雨が降っています。（todayの意）
 - 今日の若者はスマートフォンをよく使います。（nowadaysの意）

自然言語処理＞モデル＞文書の埋め込み

単語の埋め込みだけでなく，文書全体としての埋め込み表現が欲しいことも

- 文書埋め込み（document embedding），文埋め込み（sentence embedding）
 - 文書や文を，1つのベクトルにマッピング

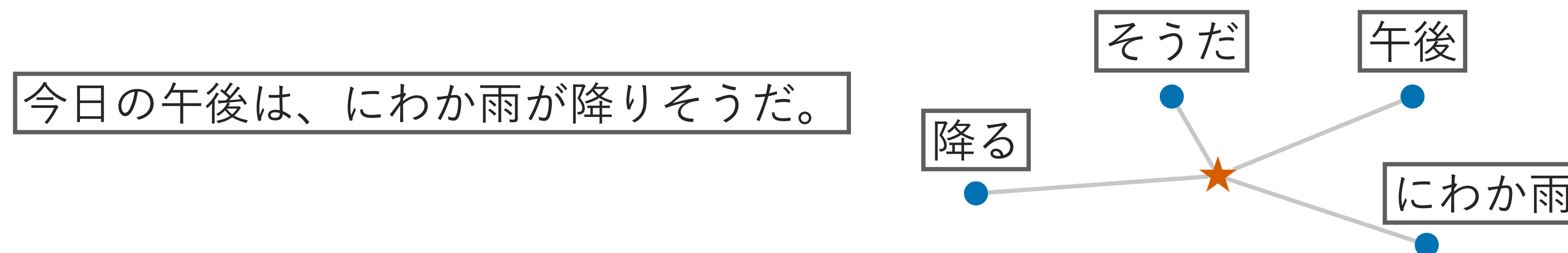


- 応用例：文書分類
 - よい埋め込み表現が得られれば，文書分類などの応用タスクに使える

自然言語処理＞モデル＞文書の埋め込み

文書中の全トークンについて，埋め込みを平均して扱うこともできそう

- 単語埋め込みで，意味論的計算がある程度できることが分かっていた
 - 全てを足して平均してしまえば，意味を混ぜた文書ベクトルとして扱えそう



- 実際に，この方針もタスクによっては有用
 - ただし単純な平均ではなく，トークンの出現頻度で重み付けする手法が良いことなどが報告されている（Arora et al., 2017）
 - 後述の，文脈を考慮する手法と比べて，計算時間が短いなどの利点はある

【補足】 単語ベクトルの平均をする手法の中では，平滑化逆頻度（smooth inverse frequency）で重み付けした和をとる手法の性能が良いことが報告されている。

【参考】 Arora et al. (2017). A simple but tough-to-beat baseline for sentence embeddings. International Conference on Learning Representations.

自然言語処理＞モデル＞文書の埋め込み

ところが、この方法では、元の並び順が持つ情報が失われてしまう

- 例えば次の文章は、単語を集めたもの以上の情報を持つ（並び順の情報が重要）

長時間待った後で、料理が出てきた瞬間の感動は大きかった。二度と行かない。

- 「感動した」という表現は後半に向けた前置きの皮肉で、全体としては批判を強める役割を持っているが、単語だけ見れば「感動」「大きい」である。

- できれば、系列としての位置関係を活かせるモデルを用いたい（事前知識）

自然言語処理＞モデル＞文書の埋め込み



単語の集合として扱うだけでは文脈情報などを適切に使えない

- 単純に平均をすると，単語の情報の寄せ集めになる
 - それでもタスクによっては有用（計算時間が短いなどの利点はある）
- 単語の並び方 = 系列としての情報も上手く使って埋め込めると望ましい
 - 文章の全体を見て出力を決めたい

自然言語処理＞モデル＞系列性

系列全体の情報を扱えるアーキテクチャが必要

- 入力としては、長さがあらかじめ決まっていないトークン列になる
 - もちろん、最初に埋め込み層に入れるので、実質的には入力はベクトルの列



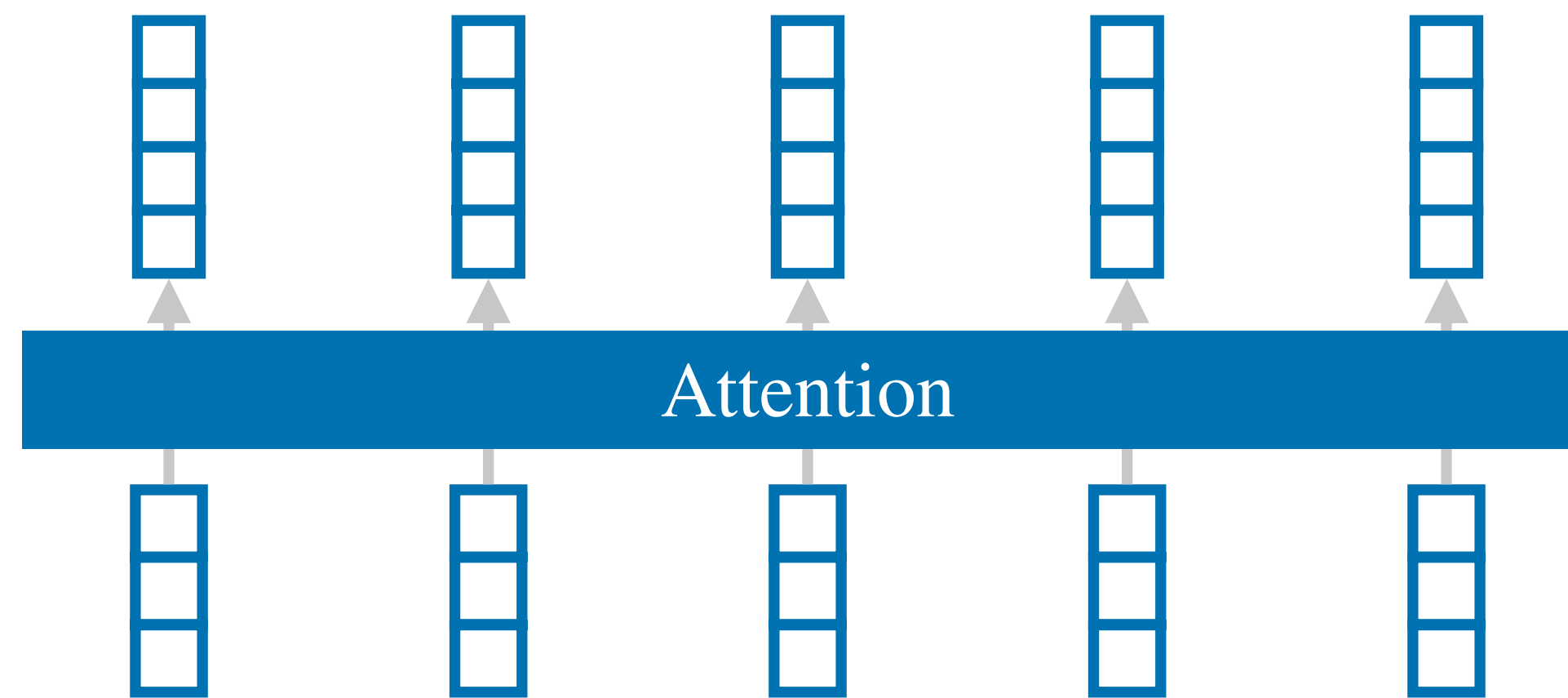
- この、可変長のベクトルを処理するレイヤーが必要
 - RNN, CNN, Transformer, といったアーキテクチャが提案されてきた

自然言語処理＞モデル＞系列性＞アテンション層

アテンションという，全体を見て出力を決める処理が現在の主流

■ アテンション層（attention layer）

- 「系列から系列」型のモデル：入力と同じ長さのベクトル列を出力する
- 処理の仕組み上，入力の長さが事前に決まっていなくても使える



- なお，系列内の位置を指すために「n番目のトークンのXX」ということがある

自然言語処理＞モデル＞系列性＞注意機構

全体を見て各トークンに対する出力値を決める処理

- アテンション，注意機構（attention）

- まず，各トークンのベクトルから，線型変換で Query, Key, Value の3つを作成
- 続いて，Query と Key の内積を使って，各トークンからの注意重みを計算
- そして，Value を重み付きで足し上げて，そのトークンでの出力とする

- これらを順に説明していく

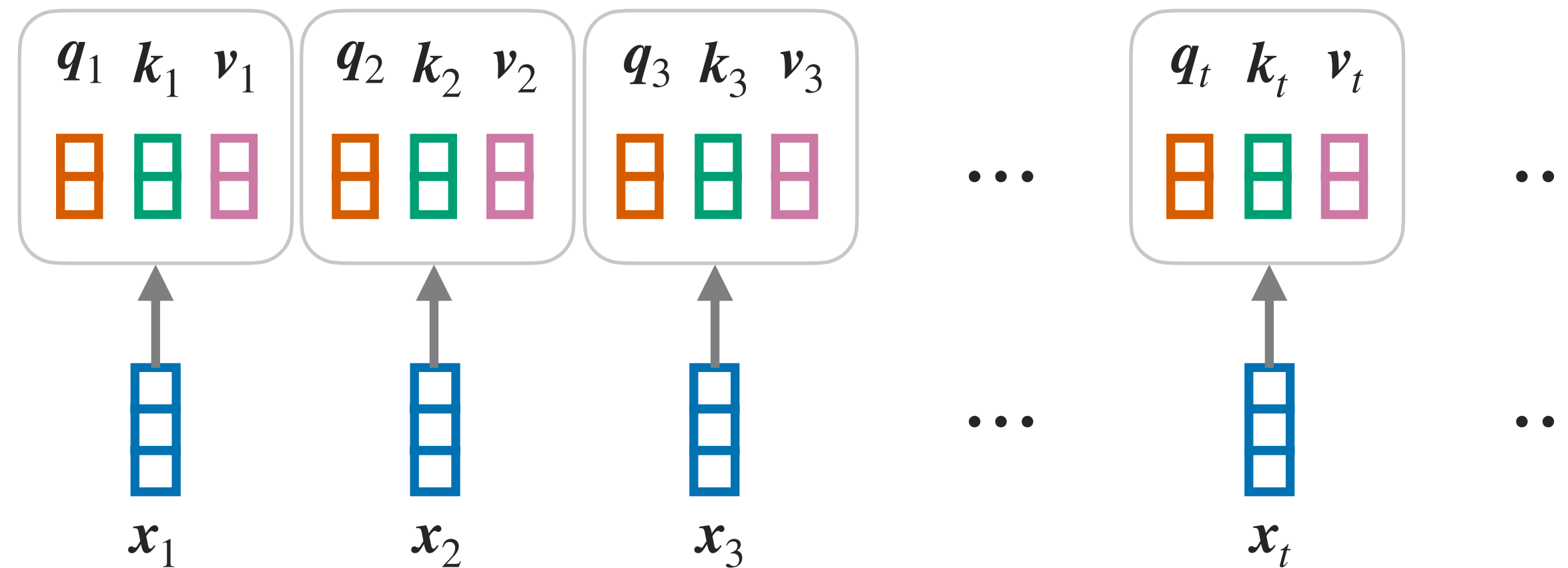
【用語】より正確には，一つの系列データ内で計算される注意機構は，特に自己注意機構（self-attention）と呼ぶ。一般に異なる系列間での注意機構を用いる場合もあり，相互注意機構（cross-attention）と呼ぶことがある。

自然言語処理＞モデル＞系列性＞注意機構＞ステップ1

クエリ・キー・バリューを計算

- まず、パラメーター $W = (W_{\text{query}}, W_{\text{key}}, W_{\text{value}})$ を、トークン埋め込みベクトルのそれぞれに適用して、トークン1つあたり3つずつのベクトルを得る

$$q_t := W_{\text{query}} \mathbf{x}_t, \quad k_t := W_{\text{key}} \mathbf{x}_t, \quad v_t := W_{\text{value}} \mathbf{x}_t$$



- クエリ・キーの次元（これらは同じである必要がある）は h としておく

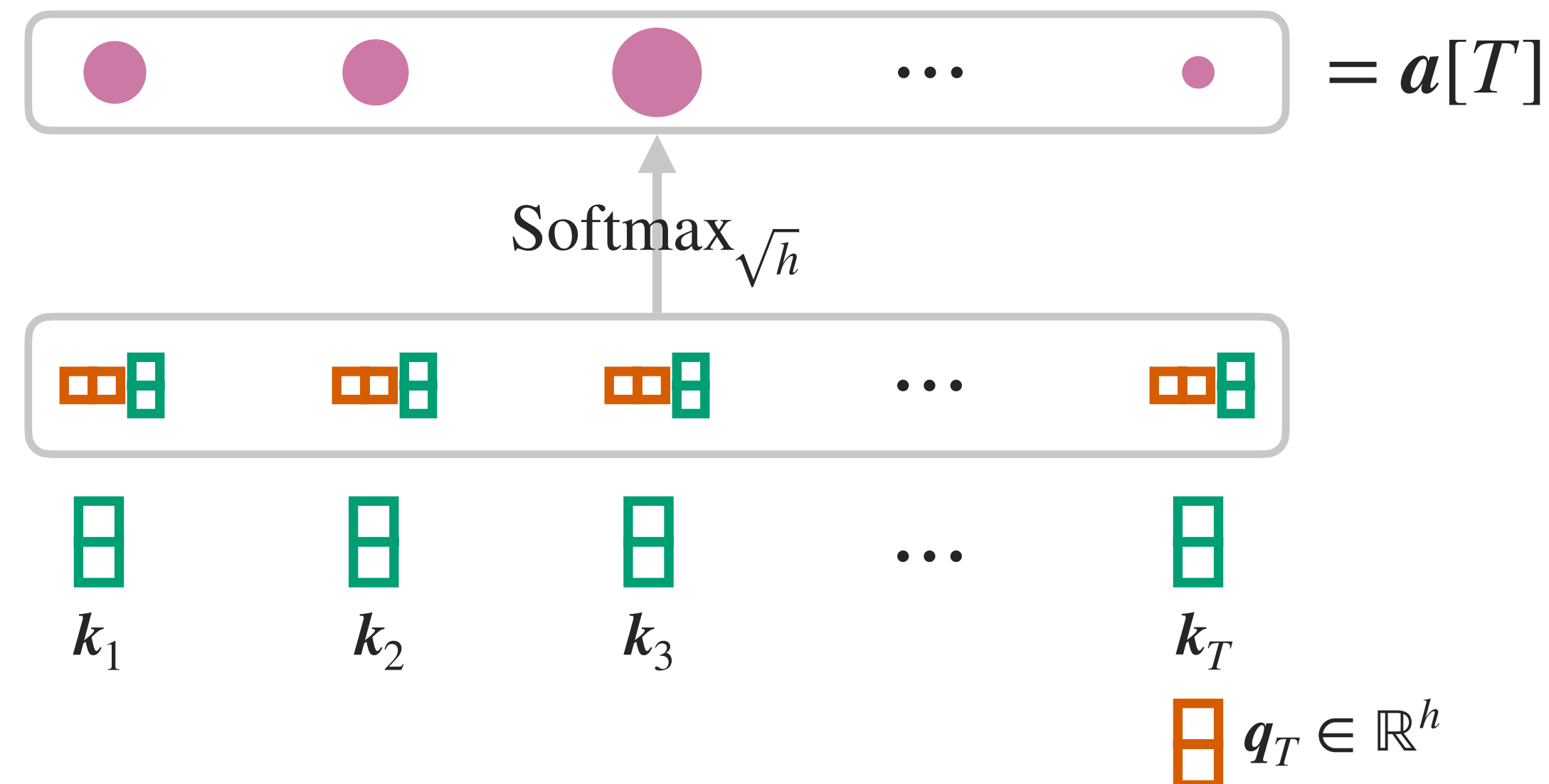
自然言語処理 > モデル > 系列性 > 注意機構 > ステップ2

各トークンごとに、自身のクエリと全体のキーから、注意重みを計算

- t 番目のトークンから他のトークンへの**注意重み (attention weights)** を計算

$$a[t] := \text{Softmax}_{\sqrt{h}} (q_t k_1, q_t k_2, \dots, q_t k_T)^\top$$

- 各 t に対してこれを計算. t 番目から s 番目への注意重みは $a[t, s]$ となる.



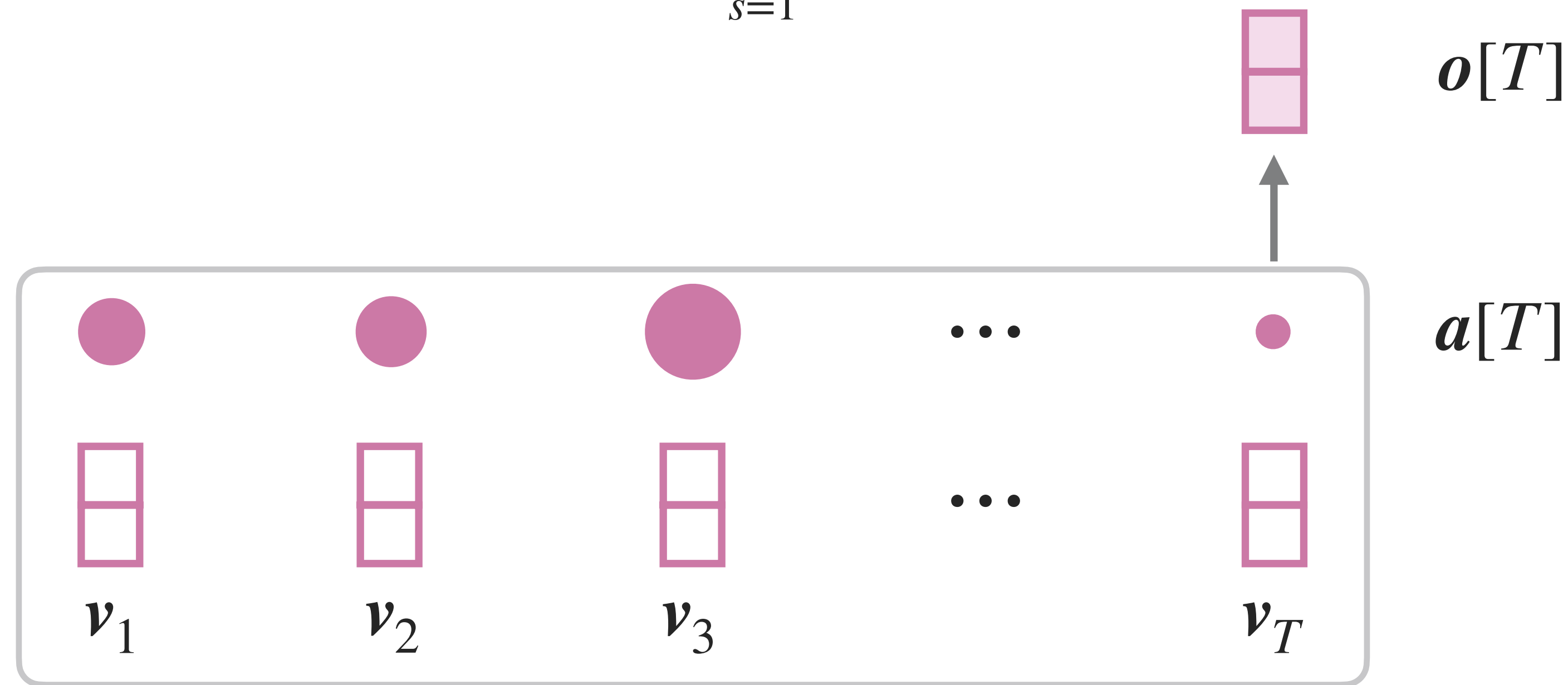
【補足】ここでは、見やすいように右端のトークンに対する計算のみを図示したが、同様に全てのトークンに関して計算が行われる。

自然言語処理＞モデル＞系列性＞注意機構＞ステップ3

バリューの重み付き和で，出力ベクトルを計算

- t 番目のトークンに対する出力値は，バリューの，注意重みによる重み付き和

$$o[t] := \sum_{s=1}^T a[t, s] \cdot v_s$$



【補足】ここでは，見やすいように右端のトークンに対する計算のみを図示したが，同様に全てのトークンに関して計算が行われる。

自然言語処理 > モデル > 系列性 > 注意機構

ここまでの式をすべて一覧すると，次のようになっている

■ 自己注意機構 SelfAttention($W, \cdot$)

- パラメーターとして，3つの行列からなる $W = (W_{\text{query}}, W_{\text{key}}, W_{\text{value}})$ をもつ
- 入力のベクトル系列 $\mathbf{x} = (\mathbf{x}_1, \dots, \mathbf{x}_T)$ を受け取る（長さは任意）
- 出力はベクトル系列 $\mathbf{o} = (\mathbf{o}_1, \dots, \mathbf{o}_T)$ （入力と同じ長さ）で，各 $\mathbf{o}_t$ は，

$$\mathbf{o}_t = \text{SelfAttention}(W, \mathbf{x})[t] := \sum_{s=1}^T a[t, s] \cdot \mathbf{v}_s$$

▶ ここで， $a[t] := \text{Softmax}_{\sqrt{h}}(\mathbf{q}_t \mathbf{k}_1, \mathbf{q}_t \mathbf{k}_2, \dots, \mathbf{q}_t \mathbf{k}_T)^\top$ はトークン t からの注意重み

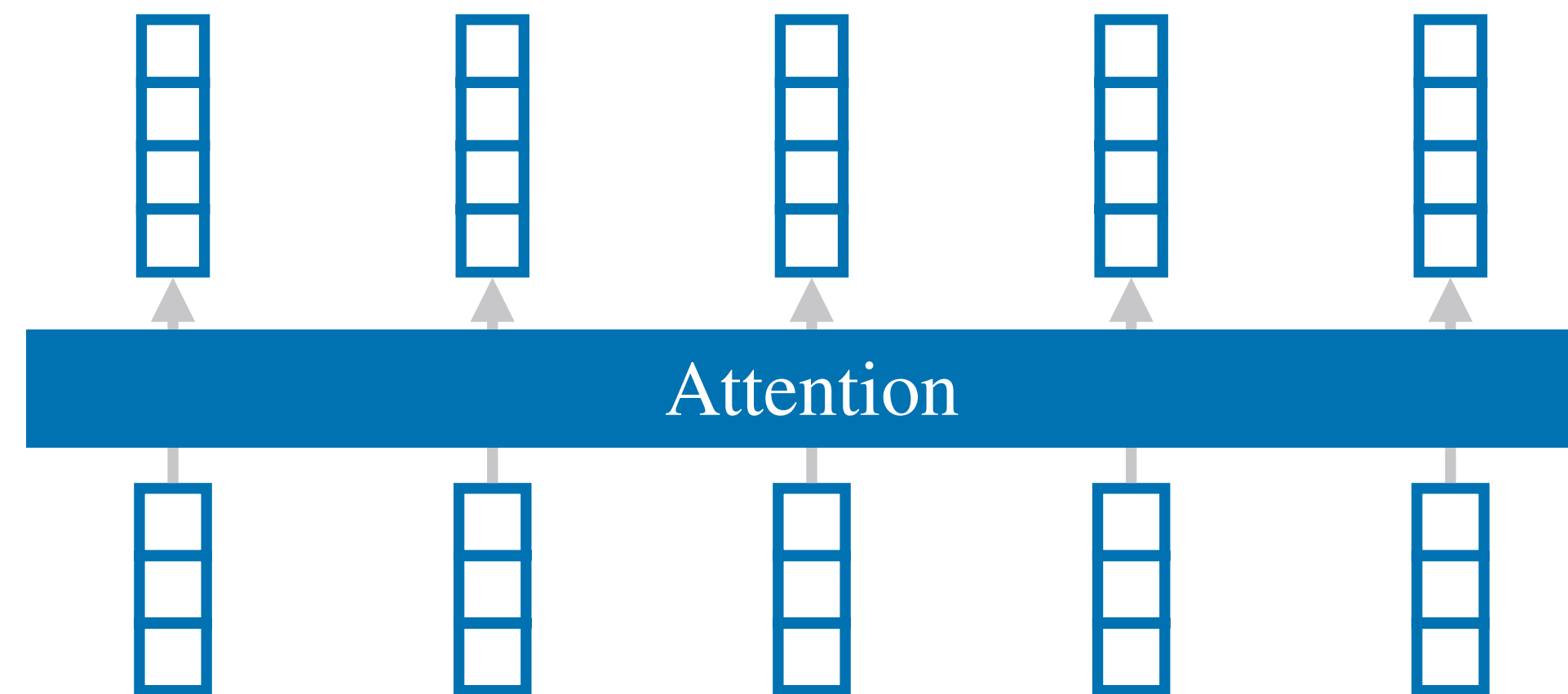
▶ また， $(\mathbf{q}_t, \mathbf{k}_t, \mathbf{v}_t)$ はそれぞれ， $\mathbf{q}_t := W_{\text{query}} \mathbf{x}_t$ ， $\mathbf{k}_t := W_{\text{key}} \mathbf{x}_t$ ， $\mathbf{v}_t := W_{\text{value}} \mathbf{x}_t$.

【補足】 ここで， h はクエリやキーの次元.

自然言語処理＞モデル＞系列性＞注意機構の役割

トークンから他のトークンの情報を参照する仕組みになっている

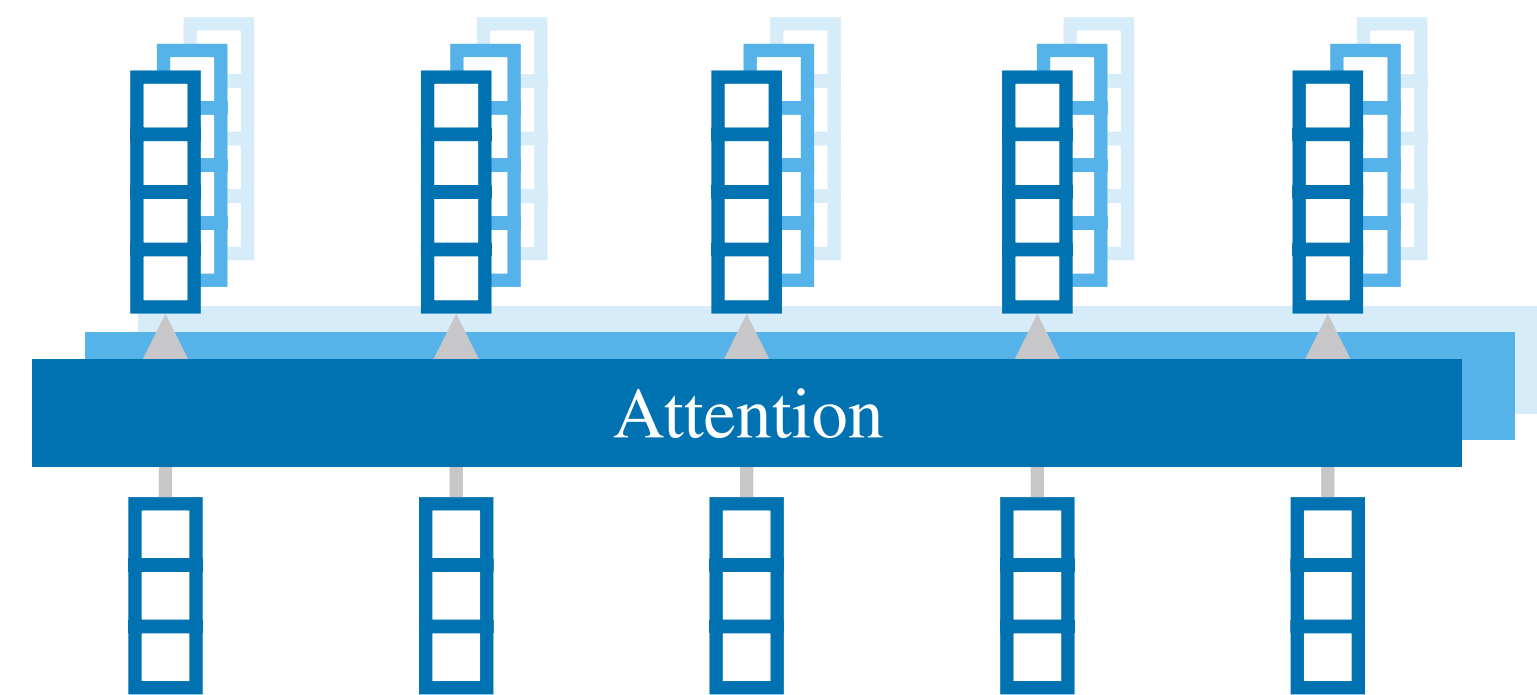
- トークン間で情報を伝え合うアーキテクチャとなっている
 - Query：私（トークン）はどんな情報を欲しているか
 - Key：私（トークン）はどんな情報を提供できるか
 - Value：私（トークン）の変換後の値



自然言語処理＞モデル＞系列性＞アテンション層

マルチヘッドアテンション層は、この注意機構を複数枚ぶん実行し、集約

- マルチヘッドアテンション層 (multi-head attention layer)
 - 複数の注意機構を用意 (パラメーターは各々別々のもの)



- 出力をトークンごとに線型層 (恒等活性化関数の全結合層) でまとめる

$$\text{MultiHeadAttention}(\{W_1, \dots, W_L\}, W_{\text{out}}, \mathbf{x})[t] := W_{\text{out}} \left(\text{SelfAttention}(W_l, \mathbf{x})[t] \right)_{l=1}^L$$

自然言語処理＞モデル＞系列性＞位置埋め込み

並び順の情報を使うために、位置埋め込みが使われる

- アテンションの計算式は、実はトークンの順番を入れ替えても結果が変わらない
 - これでは、トークン列の並び順の情報は活用できていないことになる
- 位置埋め込み（positional embedding）
 - 様々な方法が提案されているが、RoPE（rotary positional embedding）が主流
 - RoPEは、クエリとキーに、トークン位置に応じた回転行列を掛ける
 - ▶ クエリやキーのベクトルを、上から2次元ずつ組にして各々回転させる
 - ▶ 回転の角度は、クエリやキーの次元を h とし、 $\rho(r) := 10000^{-2r/h}$ と定めると、位置 i のトークンのクエリ・キーの r 組目の次元は、 $i\rho(r)$ だけ回転

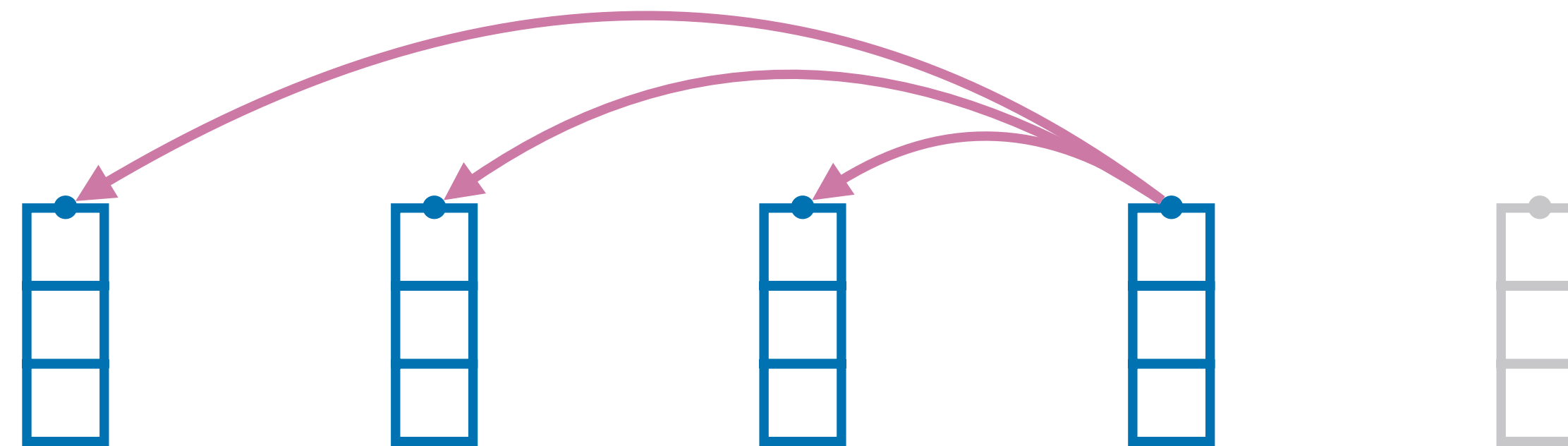
【補足】埋め込みという名前が付いているが、実際にトークンの並び位置に応じた埋め込みを学習する手法もあれば、RoPEのように新規に埋め込みのパラメーターを導入しないような手法もある。 【参考】 Su et al. (2024). RoFormer: Enhanced transformer with rotary position embedding. Neurocomputing, 568(C).

自然言語処理＞モデル＞系列性＞因果的注意機構

モデルによっては、注意重みの計算対象を自分より左のトークンに限定

■ 因果的注意機構（causal attention）

- 注意重みを計算する際に、現在のトークンより左のトークンに限り計算



- 生成タスクの場合には、通常このアーキテクチャが使われる
 - ▶ 過去の出力を条件づけて新しいトークンを出力したいため合っている

自然言語処理＞モデル＞系列性＞



実際のアーキテクチャでは更に様々なレイヤーや工夫が用いられる

- 実際には、注意機構層は「入力への加法的な差分」だけを計算（残差接続）
- アテンション層の後に、トークンごとのベクトルを順伝播層で変換
- アテンション層の前に、正規化層を適用
 - 各種の正規化層（LayerNorm, RMSNormなど）が探索されている
 - LayerNormはGroupNormの特殊ケース
 - RMSNormはLayerNormを簡易化して軽量化（平均の引き算をしない）
 - 正規化層は注意機構層や順伝播層の前が良いという知見（Xiong et al., 2020）

自然言語処理＞モデル＞系列性＞トランスフォーマー系



自然言語理解・生成ともに、現在は注意機構に基づくアーキテクチャが主流

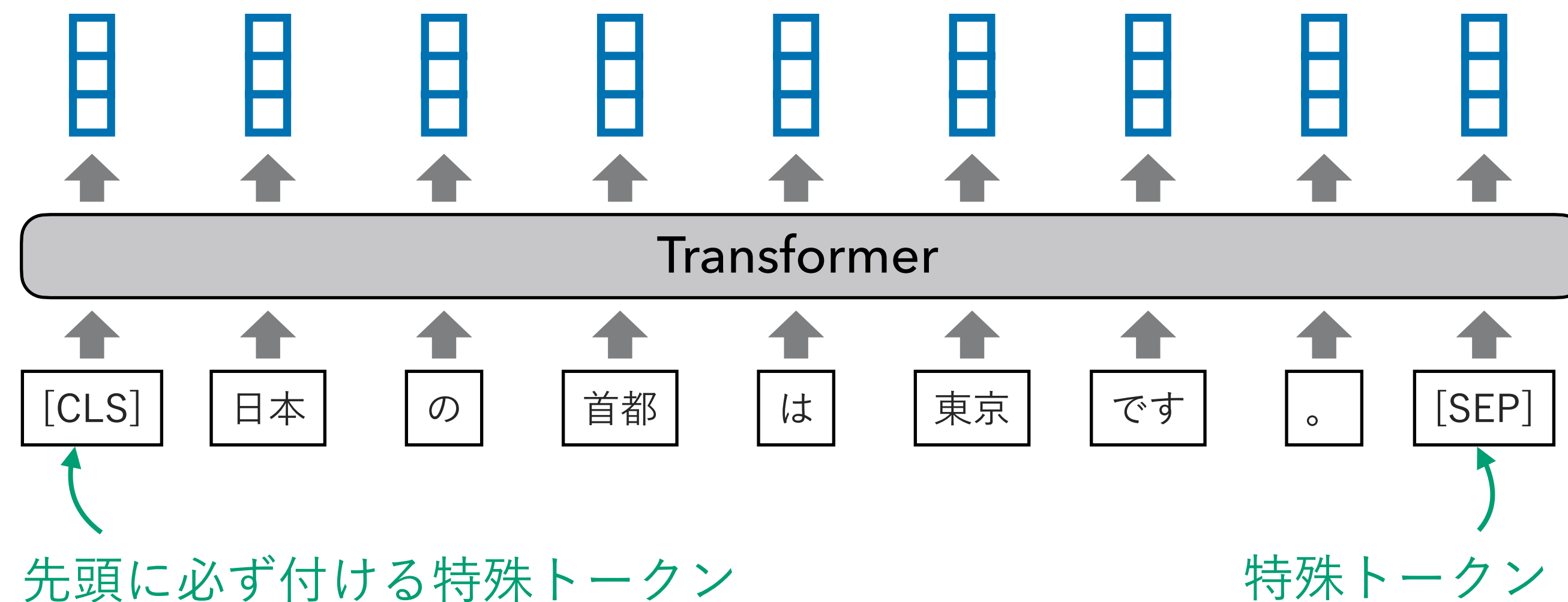
- トランスフォーマー（transformer）系のアーキテクチャという
 - 注意機構を中心に作られたアーキテクチャの総称
 - 元々は、Vaswani et al. (2017) が提案した特定のモデルの名前
- トランスフォーマー系のアーキテクチャは、大きく3つに分かれる
 - 「エンコーダー構成（encoder-only）」（BERTが代名詞）
 - 「デコーダー構成（decoder-only）」（GPTなどがある）
 - 「エンコーダー・デコーダー構成（encoder-decoder）」（T5など）

自然言語処理＞モデル＞系列性＞自然言語理解＞BERT

BERT系は「入力系列から，文脈化された表現を生成する」ことに主眼

■ BERT (bidirectional encoder representations from transformers)

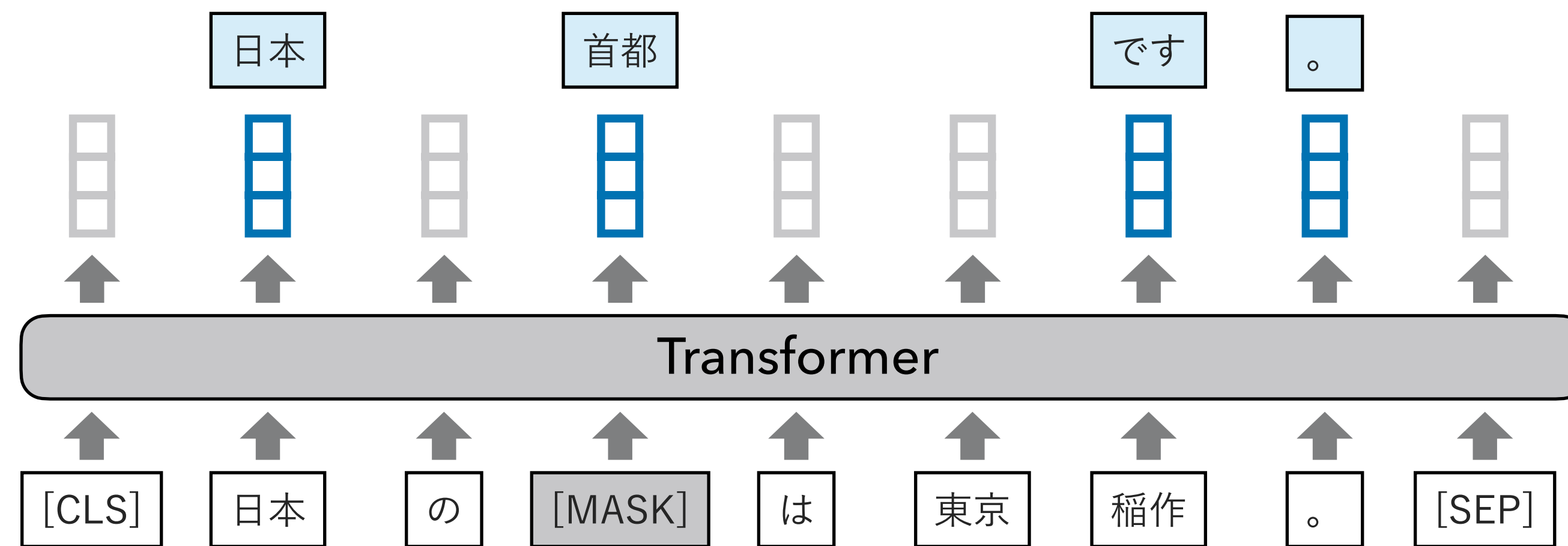
- アテンション層を多層に重ねて，系列をどんどん変換していく
- このとき，因果的注意機構にはせず，入力系列の全体を見ながら変換する



自然言語処理＞モデル＞系列性＞自然言語理解＞BERT

トークンの穴埋めをするタスクで事前学習

- マスク言語モデリング (masked language modeling) により事前学習



- 学習対象トークンを15%ランダムに選択→入力にランダムに以下の加工を行う
 - ▶ [MASK] トークンに置換, 原文ママ, ランダムなトークンに置換
- 学習対象トークンから, 原文のトークンを正解ラベルとして予測

【補足】 訓練時は, 各トークンへの出力ベクトルから確率値ベクトルに変換するレイヤーを付け加えて, 語彙をクラス集合とする確率論的分類の学習を行う。

自然言語処理＞モデル＞系列性＞自然言語理解＞BERT

トークンの穴埋めタスクで何を学習できるか？

■ 常識的な知識から文法的な知識まで

- 「日本の首都は東京です。」 「日本の首都は東京です。」 「1ダースは12本。」
- 「日本の首都は東京です。」 「日本の首都は東京です。」

■ 名詞の対応を解析する共参照解析

- 「遥さんは図書館で本を借り、それを読みました。」

■ 感情分析

- 「長時間待った後で、料理が出てきた瞬間の感動は大きかった。二度と行かない。」

■ 算術演算：「光さんは100円を持っています。38円の鉛筆を買ったので残りは62円です。」

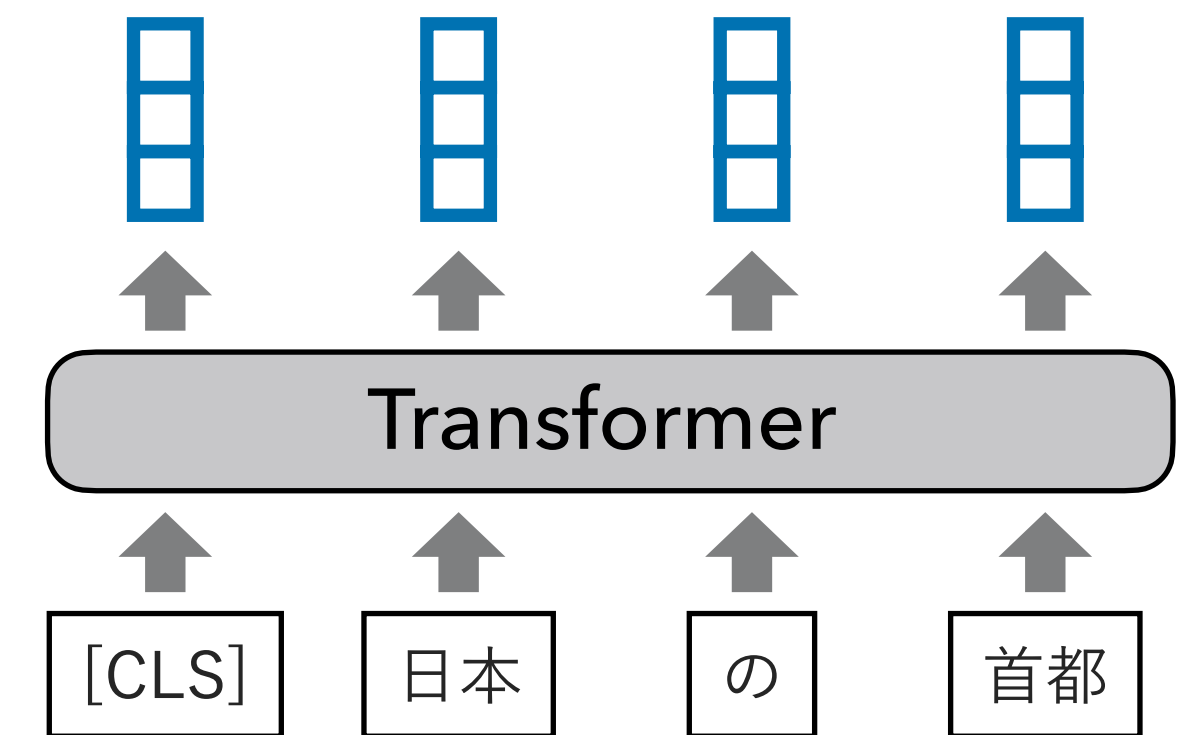
■ 推論：「午前中から会議続きだったので、ランチを食べ損ねた。」

【補足】人が「読みました」を予測するときは、「それ」が「本」を指すことを使っている。

自然言語処理＞モデル＞系列性＞自然言語理解＞BERT

BERT系のモデルによる埋め込みの使い方

- 下流タスクに合わせたレイヤーを追加して全体を精密調整
- 文書分類タスクに使う場合
 - 特殊トークン[CLS]の出力ベクトルを使う（全結合層＋ソフトマックス）
- 系列ラベリングタスクに使う場合
 - 各トークンの出力ベクトルを使う（全結合層＋ソフトマックスなど）

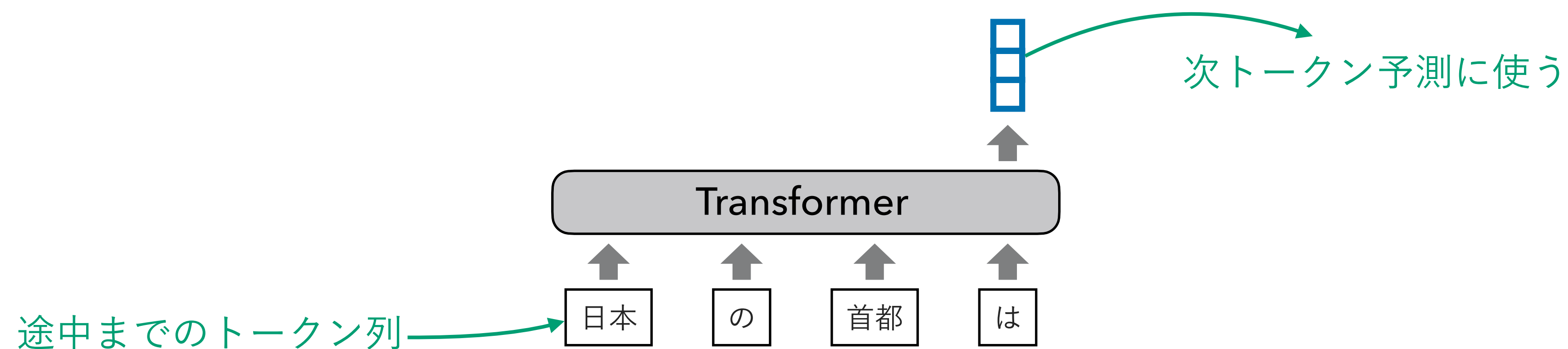


自然言語処理＞モデル＞系列性＞自然言語生成＞GPT

GPT系は「途中まで与えられたトークン列から続きを生成する」ことに主眼

■ GPT (generative pre-trained transformer)

- 因果的注意機構を用いて、系列をどんどん変換していく
- 入力されたトークン列の次のトークンの条件付き確率を学習させる
- その予測には、最終トークンに対応する出力ベクトルを使う

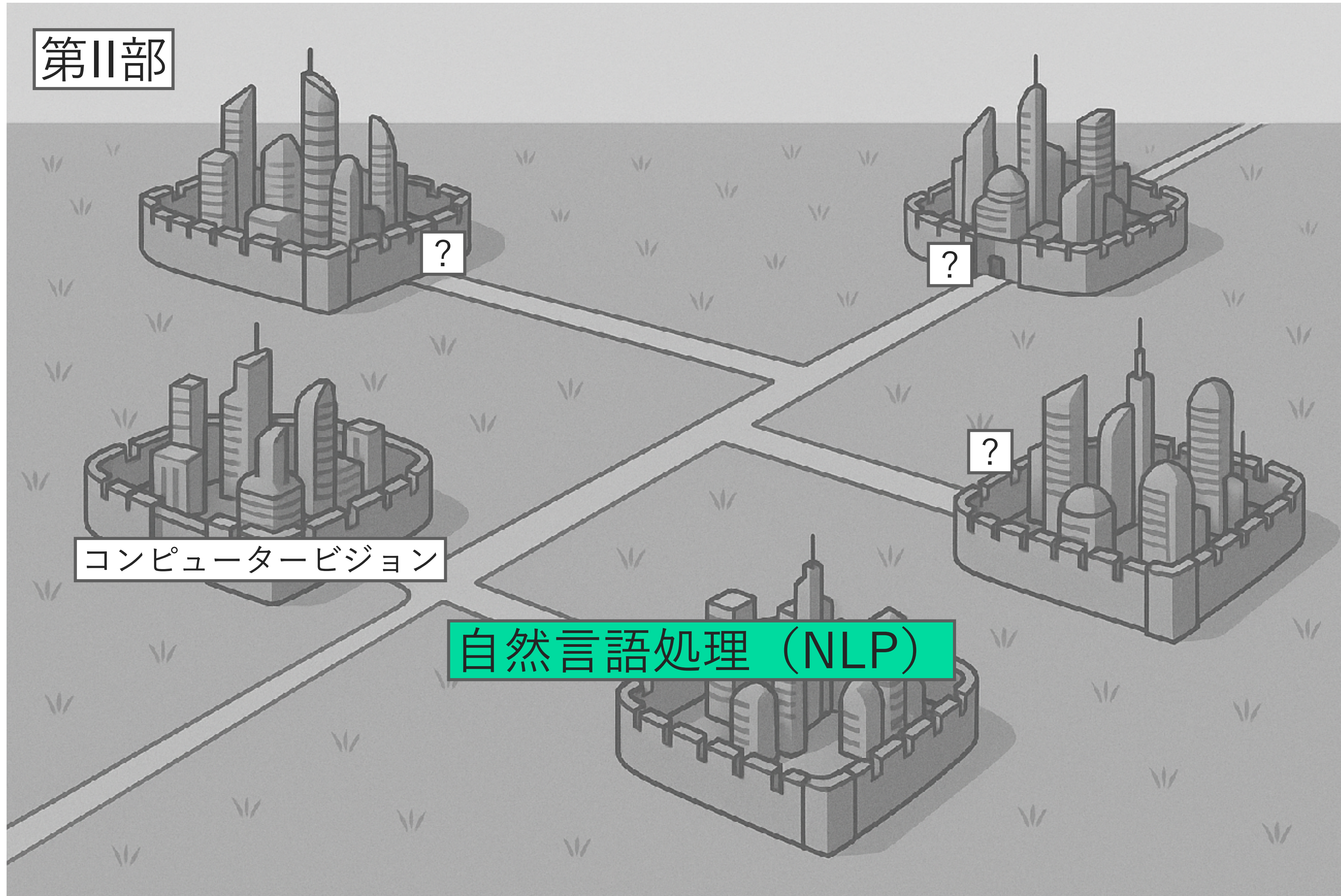


自然言語処理＞モデル＞系列性



系列生成の学習については生成タスクの学習のセクションで解説します

ロードマップ



ロードマップ

